

A Project Report

On

L.E.A.R.N: Learning Enhanced by AI & Reasoning

Submitted by

Aditya Sambhaji Jadhav

B Ankit

Sriharsha Vemuri

Yash Rathi

Supervised by

Prof. Sara Hooshangi

Contents

List of Figures	iii
1 Product Description	1
2 Product Functionalities	2
2.1 Landing on the Home Page	3
2.2 Returning Users: Login Page	4
2.3 New Users: Sign-Up Page	4
2.4 User Dashboard: Progress Tracking and Personalization	5
2.5 Topic Search Interface with Language Selection	5
2.6 Content and Relevance Moderation	6
2.7 Generation of Contents: Dynamic Table of Contents (ToC)	7
2.8 Engaging Users During Lesson Generation	8
2.9 Generation of the First Lesson	9
2.10 Interactive Elements	10
2.11 Chatbot Implementation	11
2.11.1 L.E.A.R.N Mode: Guided Lesson Walkthrough	11
2.11.2 Clarify Mode: Term or Sentence Explanation	12
2.11.3 Questions Mode: Open-Ended Exploration	13
2.12 Adaptive Quizzes for Personalized Assessment	14
3 Design	15
3.1 Architecture Diagram	16
3.2 Core Functionality and Implementation	18
3.2.1 Dynamic Lesson Generation	18
3.2.2 RAG-Based Factual Checking	18
3.2.3 Multilingual Support	19
3.2.4 Text-to-Speech (TTS)	20
3.2.5 Interactive Learning Elements	20
3.2.6 Content and Relevance Moderation	21
3.2.7 Trivia and Image Generation	21
3.2.8 Chatbot Integration	22
3.3 System Design and Implementation	23
3.3.1 System Design Overview	23
3.3.2 Algorithms and AI Integration	23
3.3.3 Framework Implementation	24
3.3.4 Backend Design and Uniqueness	25
3.3.5 Unique Features and Design Decisions	25
3.3.6 Design Decisions Leading to Uniqueness	25

4	Retrospection	27
5	Recommendations for future	29
5.0.1	Team Process Learnings	30
5.0.2	Product Expansion Goals	30

List of Figures

2.1	Landing page view when a user enters the platform.	3
2.2	Login page for returning users to securely access their personalized learning dashboard.	4
2.3	Sign-up interface for new users joining the platform.	4
2.4	User dashboard showing personalized learning progress and lesson tracking.	5
2.5	Revamped topic search interface with integrated language selection.	5
2.6	System-generated prompt shown when a submitted topic fails the relevance or content moderation check.	6
2.7	Backend response triggered by moderated or inappropriate input	6
2.8	Dynamically generated Table of Contents with language toggle support. . .	7
2.9	Trivia facts and image are shown while the lesson loads.	8
2.10	Trivia facts and image are shown while the lesson loads.	8
2.11	Generated first lesson with multilingual text and audio support via text-to-speech.	9
2.12	Dynamically generated quizzes and visual aids presented in the user's chosen language.	10
2.13	Chatbot in L.E.A.R.N mode providing a simplified walkthrough of the lesson.	11
2.14	Chatbot in Clarify mode offering deeper explanation for a selected sentence.	12
2.15	Chatbot in Questions mode answering a custom follow-up query from the user.	13
2.16	Adaptive quiz adjusting question difficulty based on user performance. . .	14
2.17	Adaptive quiz adjusting question difficulty based on user performance. . .	14
3.1	System architecture of the L.E.A.R.N platform.	16

Chapter 1

Product Description

In today's world, students and self-learners alike are overwhelmed by rigid, one-size-fits-all learning styles that often fail to adapt to individual learning needs. Whether it's a high school student struggling with math concepts, a parent seeking structured content for homeschooling, or a retiree wanting to explore philosophy without complex interfaces, traditional tools while comprehensive often fall short. That's why we built L.E.A.R.N i.e. Learning Enhanced by AI and Reasoning. L.E.A.R.N is an AI-powered interactive, learning companion that complements the existing learning approaches - from the classroom to online learning platforms. L.E.A.R.N aims to address their shortcomings without entirely replacing them. L.E.A.R.N achieves that by being tailor-able and focusing on accessibility to people from all walks of life and thus personalizing the user's learning journey. Our platform dynamically generates lesson content tailored to each learner's pace, adapts quizzes in real time based on performance, and makes learning fun through interactive elements like flashcards, trivia, and graphs, among others. Users can simply type what they want to learn. Anything, Anywhere, Any topic. No need to go through convoluted online searching, visit multiple subreddits or even master complex prompts. With multilingual support across 32 languages and real-time text-to-speech support, it ensures inclusive access for non-native speakers and users with reading difficulties. Features like conversational chatbot address real challenges, such as learners getting stuck on tough concepts or lacking the confidence to ask for help. And with generated lessons backed by trusted sources like Wikipedia, PubMed, and ArXiv, learners get not just tailored content, but credible information. L.E.A.R.N aims to transform static, monotonous study into smart study - bridging gaps, breaking barriers, and putting accessible and personalized education on any topic into every learner's hands.

[View Demo](#)

Chapter 2

Product Functionalities

This section walks through the user experience on our platform using a series of screenshots and narrative descriptions. Each stage of the platform interaction is illustrated with visuals to provide a clear understanding of its core functionalities.

2.1 Landing on the Home Page

Imagine a user, curious to learn something new, arrives at our platform. They are greeted with a clean and inviting home page that introduces the platform's capabilities and invites them to begin exploring.

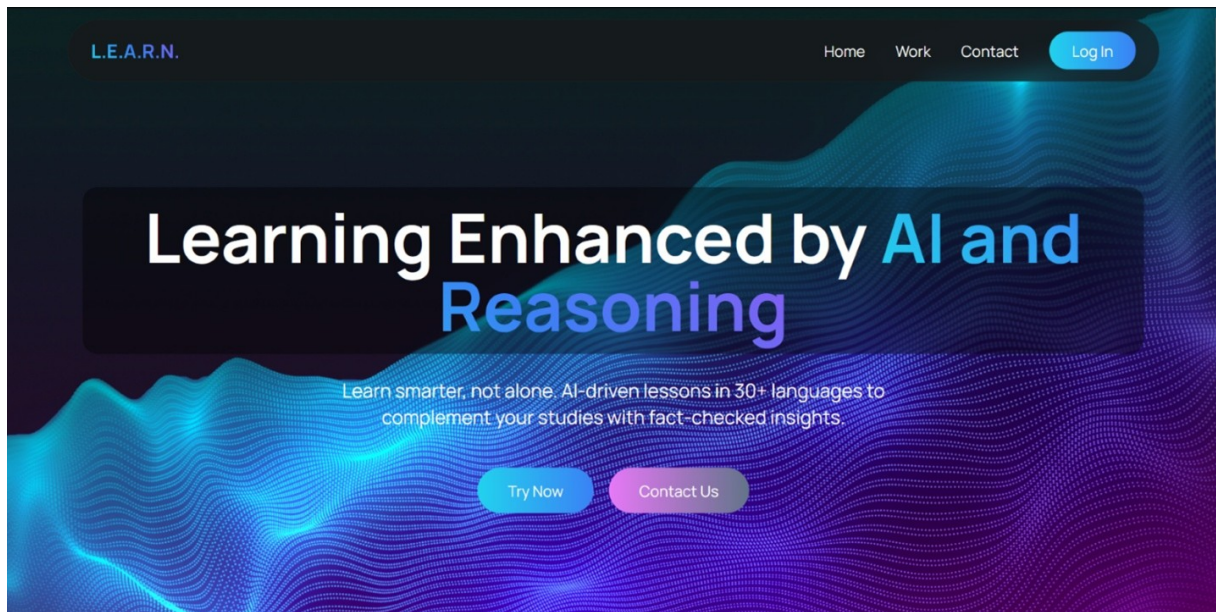


Figure 2.1: Landing page view when a user enters the platform.

2.2 Returning Users: Login Page

Returning users are directed to a simple and intuitive login page. Here, they can quickly enter their credentials and pick up right where they left off. This helps maintain user continuity and progress tracking across sessions.

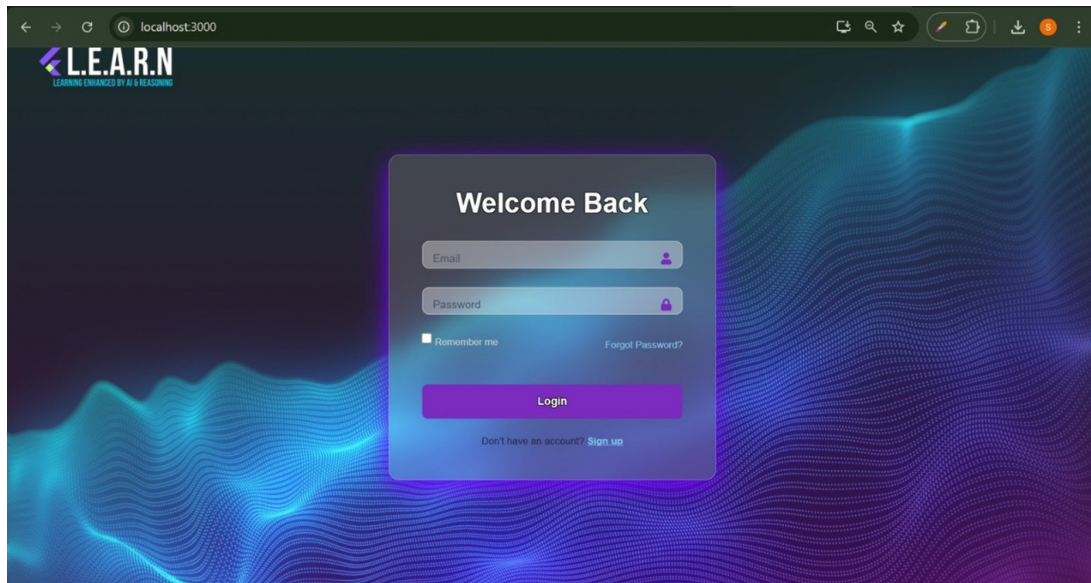


Figure 2.2: Login page for returning users to securely access their personalized learning dashboard.

2.3 New Users: Sign-Up Page

For new users, the platform offers a streamlined sign-up process. The sign-up form collects only essential information, ensuring minimal friction while still enabling personalized learning experiences based on tracked progress.

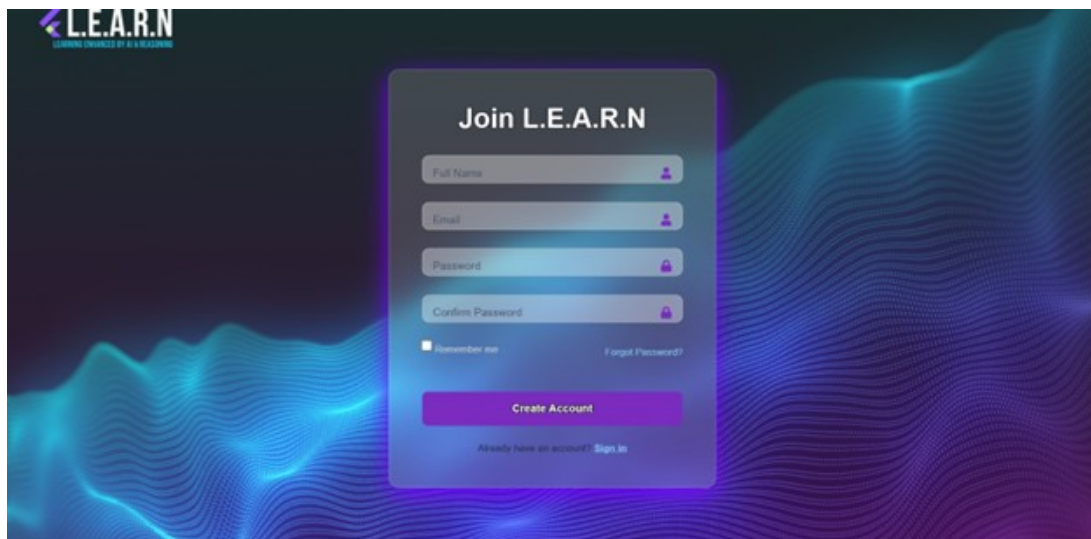


Figure 2.3: Sign-up interface for new users joining the platform.

2.4 User Dashboard: Progress Tracking and Personalization

Once logged in, the user is taken to a personalized dashboard that gives a snapshot of their learning progress. This dashboard displays completed lessons, ongoing modules, and quiz performance metrics. It is designed with clarity and usability in mind, enabling learners to quickly understand where they left off and what they should focus on next.

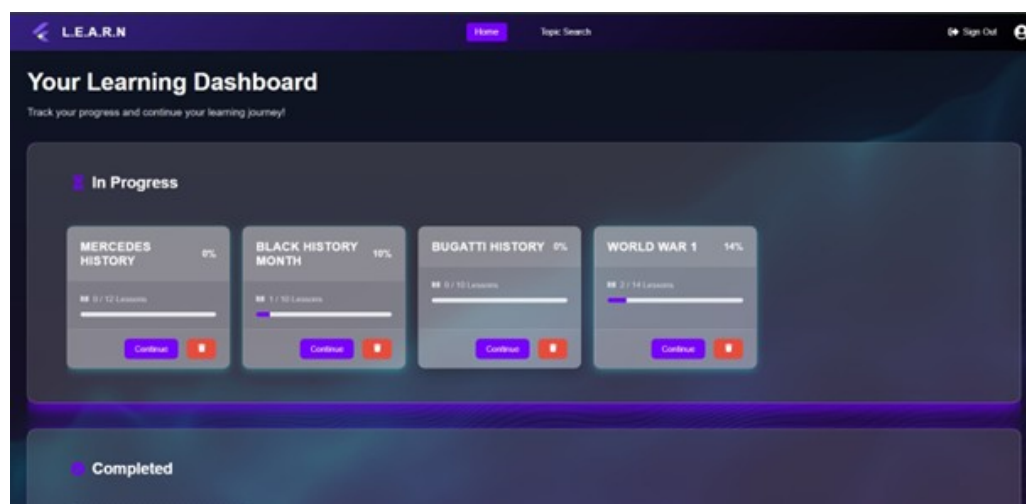


Figure 2.4: User dashboard showing personalized learning progress and lesson tracking.

2.5 Topic Search Interface with Language Selection

From the dashboard or home page, users can explore new topics using the topic search interface. This revamped UI allows users to either type in a topic of their choice or explore suggestions generated by the system. Additionally, users can select their preferred language from a dropdown, making the platform fully multilingual and accessible to a global audience.

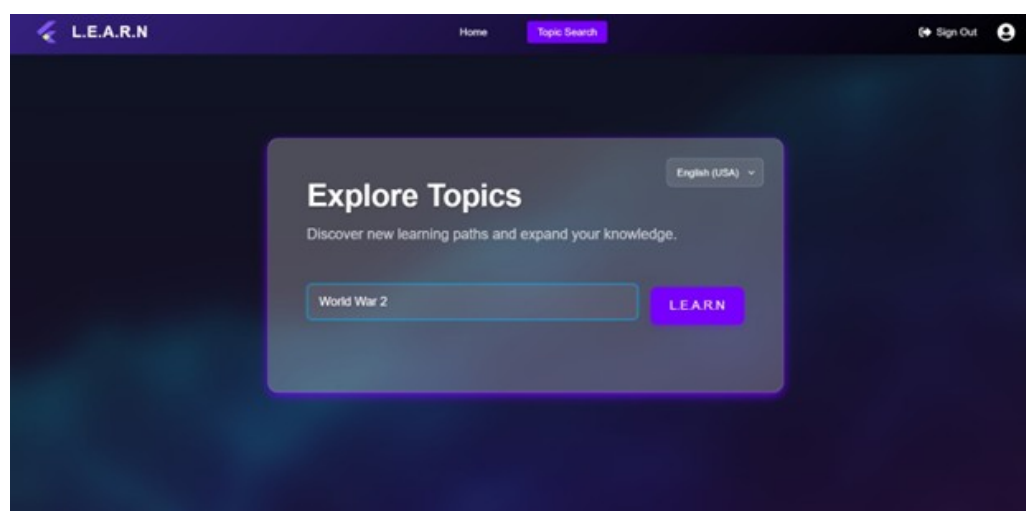


Figure 2.5: Revamped topic search interface with integrated language selection.

2.6 Content and Relevance Moderation

To ensure that the platform remains focused on delivering safe, educational, and meaningful content, every user-submitted topic undergoes an automatic moderation check. This validation step helps filter out inappropriate, irrelevant, or non-educational prompts before they reach the lesson generation pipeline.

For example, if a user inputs a topic that falls outside supported academic or historical domains, the system immediately flags it and suggests refining the topic. This feedback is provided in a friendly, non-disruptive manner, allowing the user to stay engaged without confusion or frustration.

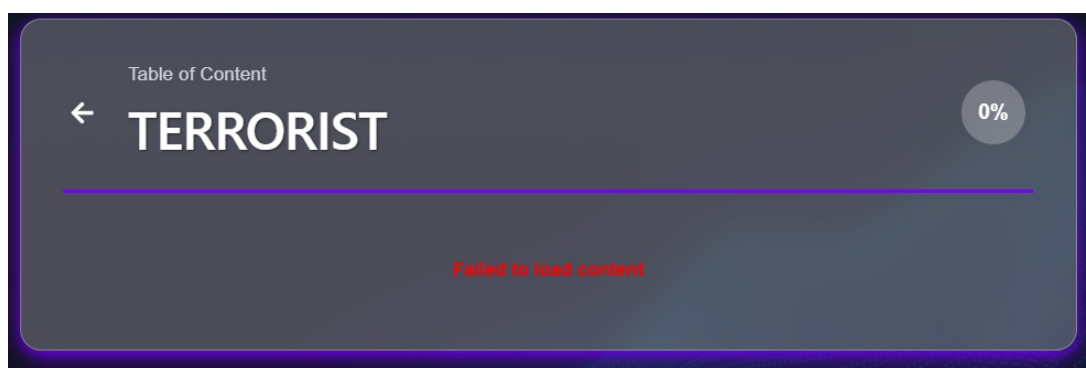


Figure 2.6: System-generated prompt shown when a submitted topic fails the relevance or content moderation check.

Server response	
Code	Details
400 <i>Undocumented</i>	Error: Bad Request Response body <pre>{ "detail": "Topic rejected by moderation: The topic contains a vulgar and offensive word." }</pre>

Figure 2.7: Backend response triggered by moderated or inappropriate input

2.7 Generation of Contents: Dynamic Table of Contents (ToC)

Once a user submits a topic of interest, the platform dynamically generates a structured Table of Contents (ToC) tailored to that subject. This ToC outlines a suggested learning path, breaking the topic down into multiple, logically sequenced lessons. To reduce cognitive overload and keep users focused, only the first lesson is unlocked by default. This encourages linear progression and supports the platform's adaptive learning goals.

At this stage, users can also switch to their preferred language. When a different language is selected, the entire Table of Contents is instantly translated and re-rendered in that language, providing a seamless multilingual experience. This inclusive design allows learners from diverse backgrounds to access structured learning content in a way that feels natural and personalized.

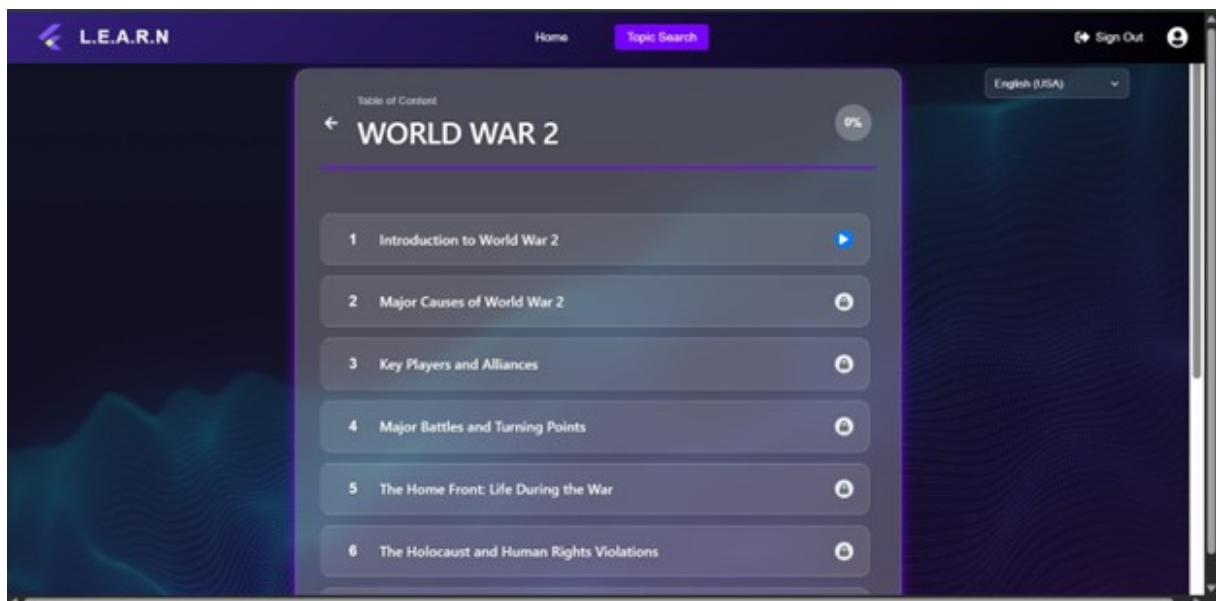


Figure 2.8: Dynamically generated Table of Contents with language toggle support.

2.8 Engaging Users During Lesson Generation

To maintain user engagement during the short delay while a lesson is being generated, the platform displays a curated set of trivia facts alongside a topic-relevant AI-generated image. Rather than showing a generic loading animation, this screen provides a fun, low-pressure introduction to the subject. The trivia is designed to spark curiosity and offer bite-sized, interesting facts, while the image visually primes the learner for the content ahead.



Figure 2.9: Trivia facts and image are shown while the lesson loads.



Figure 2.10: Trivia facts and image are shown while the lesson loads.

2.9 Generation of the First Lesson

Once the user selects the initial lesson from the dynamically generated Table of Contents, the system immediately begins generating the full lesson content on the fly. This real-time generation ensures that the lesson is tailored to the user's chosen topic, structured using verified sources via RAG, and presented in the user's preferred language.

The lesson interface is designed to be clear, accessible, and interactive. At the top of the lesson view, users can see the selected language applied to all content elements. A built-in speaker icon allows users to listen to the lesson using the platform's real-time text-to-speech (TTS) system, powered by Eleven Labs. This feature is particularly useful for auditory learners and users with visual or reading challenges.

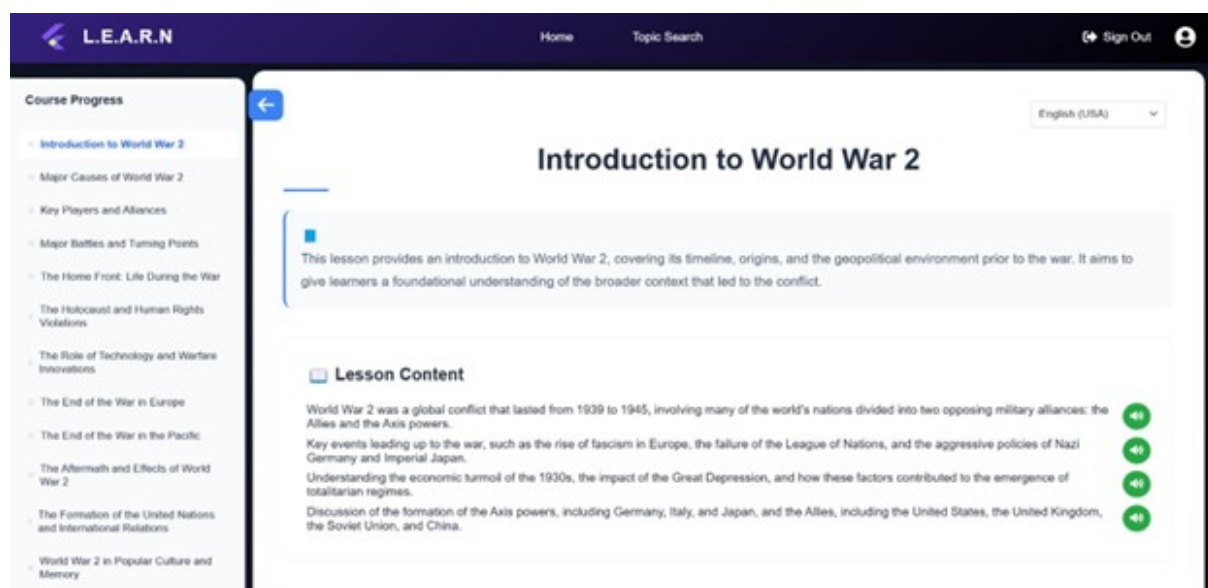


Figure 2.11: Generated first lesson with multilingual text and audio support via text-to-speech.

2.10 Interactive Elements

To make the learning experience more dynamic and participatory, the platform integrates a wide range of interactive elements directly into each lesson. These include multiple-choice quizzes, flashcards, drag-and-drop activities, intuitive graphs, and visual timelines. Each of these elements is generated on the fly based on the lesson content, and fully rendered in the user's selected language to ensure accessibility and inclusiveness.

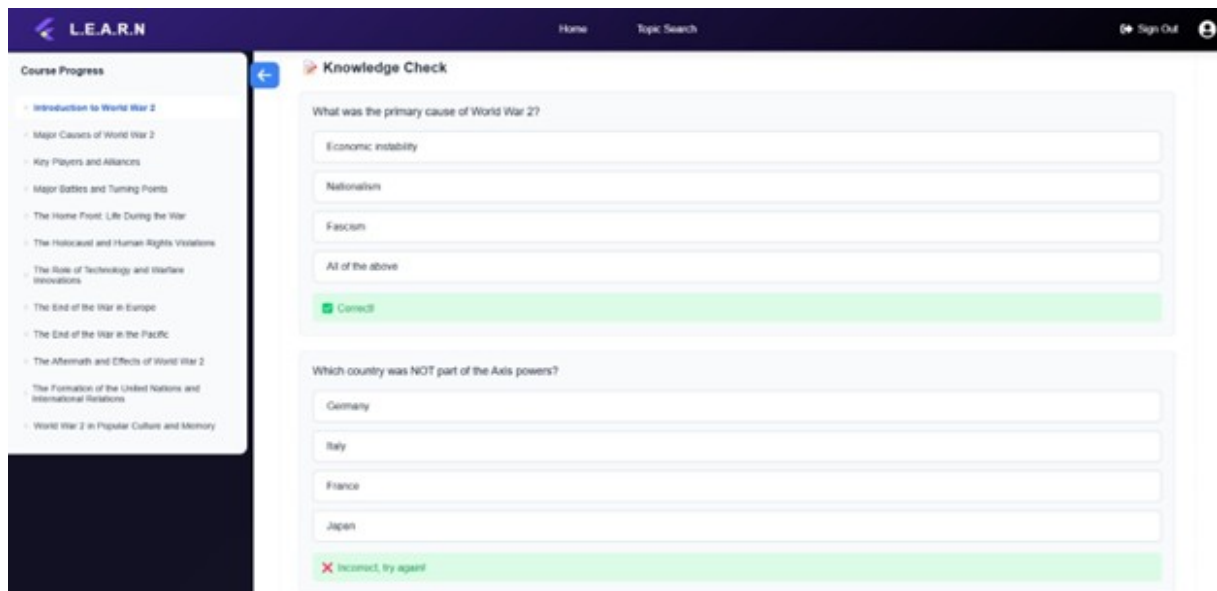


Figure 2.12: Dynamically generated quizzes and visual aids presented in the user's chosen language.

2.11 Chatbot Implementation

To assist users throughout their learning journey, the platform features an intelligent, multilingual chatbot that provides real-time support directly within each lesson. This chatbot is context-aware and can be invoked by simply right-clicking on any sentence or word that the user finds difficult to understand. It is tightly integrated with the lesson content and operates in three distinct modes: **L.E.A.R.N**, **Clarify**, and **Questions**.

2.11.1 L.E.A.R.N Mode: Guided Lesson Walkthrough

In L.E.A.R.N mode, the chatbot offers a simplified, structured walkthrough of the current lesson. It breaks down complex explanations into easy-to-understand segments, helping the user absorb key ideas gradually. This mode is especially helpful for new learners or those who prefer guided exploration over reading long paragraphs at once.

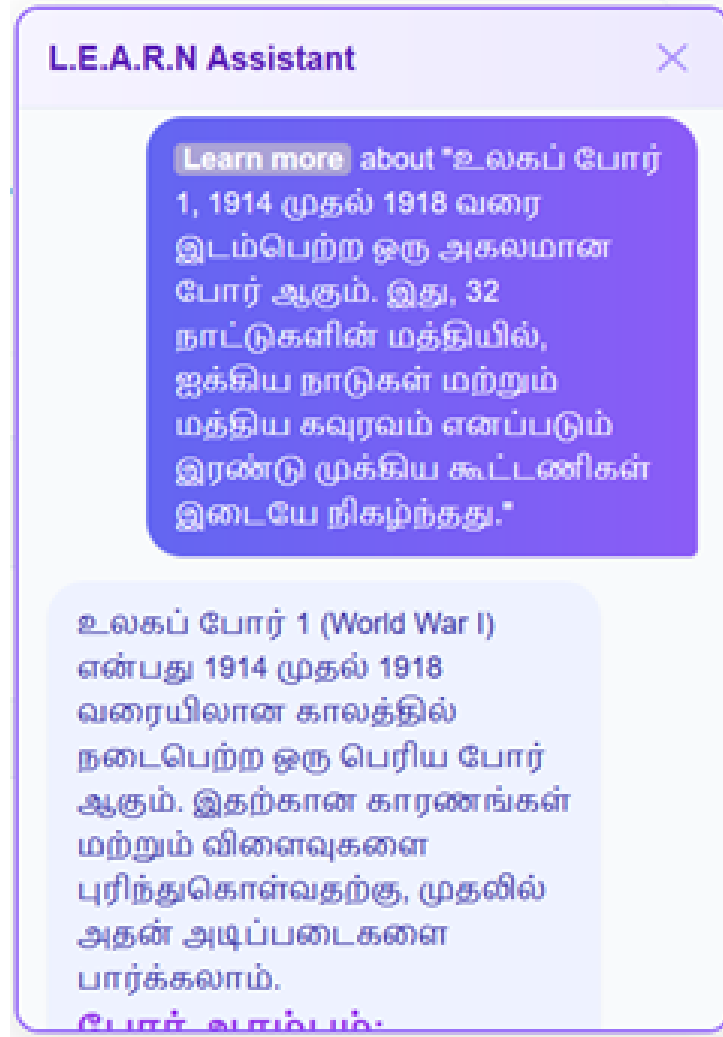


Figure 2.13: Chatbot in L.E.A.R.N mode providing a simplified walkthrough of the lesson.

2.11.2 Clarify Mode: Term or Sentence Explanation

When the user highlights a specific term, sentence, or concept, the chatbot enters Clarify mode. It explains the selected content in more detail—offering definitions, analogies, and background context. This is especially useful for understanding difficult terminology, historical references, or domain-specific knowledge.

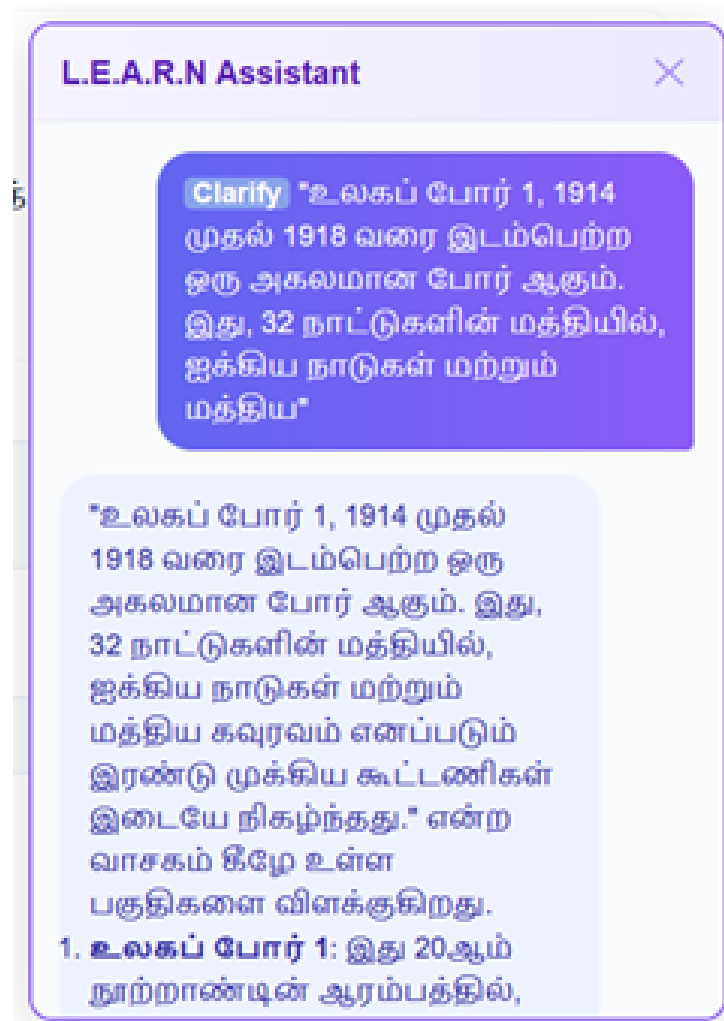


Figure 2.14: Chatbot in Clarify mode offering deeper explanation for a selected sentence.

2.11.3 Questions Mode: Open-Ended Exploration

In Questions mode, the chatbot allows users to ask free-form questions related to the topic. Whether it's a follow-up inquiry, a "why" question, or something tangentially related to the lesson, the chatbot responds in a context-aware and conversational manner. This mode encourages curiosity and deeper understanding while supporting exploratory learning.

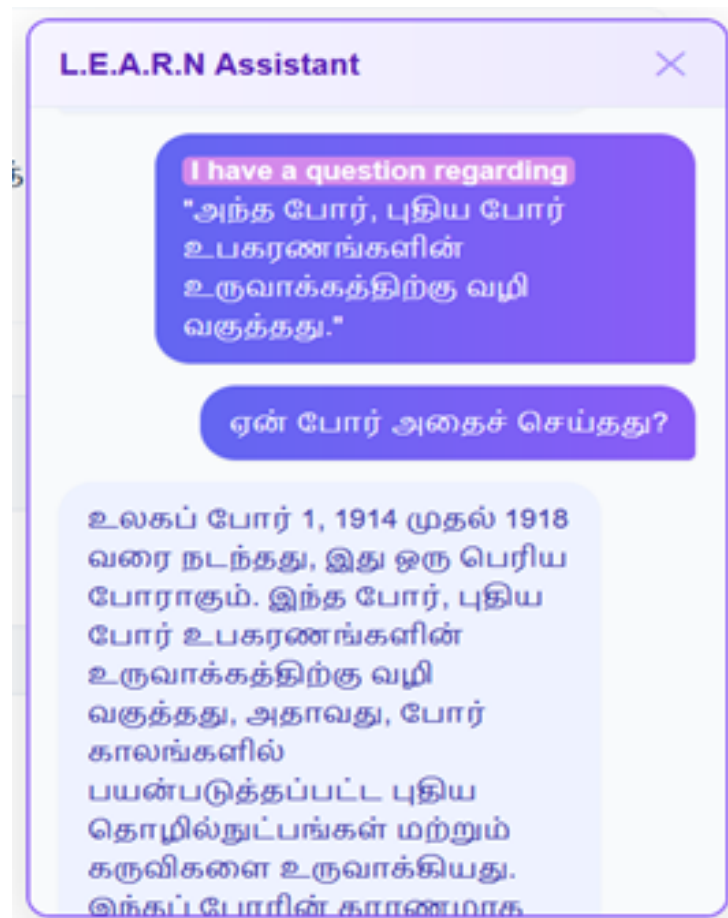


Figure 2.15: Chatbot in Questions mode answering a custom follow-up query from the user.

2.12 Adaptive Quizzes for Personalized Assessment

To personalize the learning experience and reinforce retention, the platform integrates adaptive quizzes that adjust in real time based on user performance. Each quiz begins with foundational questions to gauge the learner's baseline understanding. As the user answers correctly, the system dynamically introduces more complex questions to challenge them further. Conversely, if a user struggles, the quiz adjusts by presenting simpler, more scaffolded questions to build confidence and reinforce core concepts.

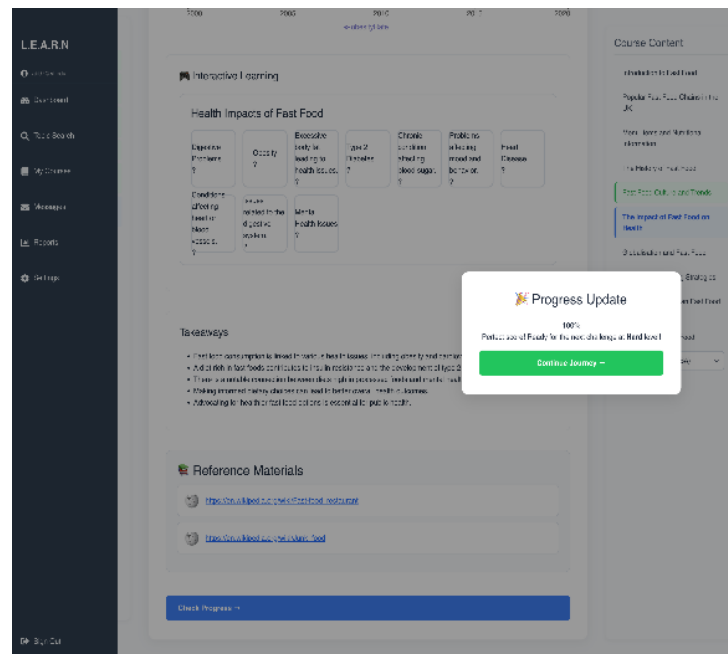


Figure 2.16: Adaptive quiz adjusting question difficulty based on user performance.

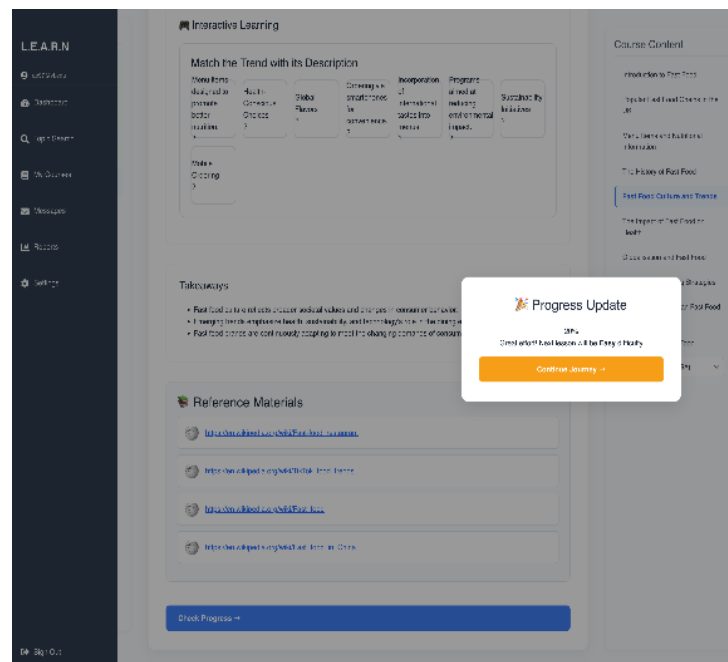


Figure 2.17: Adaptive quiz adjusting question difficulty based on user performance.

Chapter 3

Design

Our project follows a modular full-stack architecture combining Spring Boot and FastAPI for backend development, React.js for the frontend, and a PostgreSQL database to store user data, real-time generated table of contents, progress, and AI-generated lesson content. The system is designed to provide personalized, adaptive and interactive learning experiences with persistent tracking of user engagement and progress.

3.1 Architecture Diagram

The architecture of the L.E.A.R.N platform is designed around modular, scalable services that interact via RESTful APIs. Each component is independently responsible for a key part of the system’s functionality, ensuring clear separation of concerns and ease of maintenance.

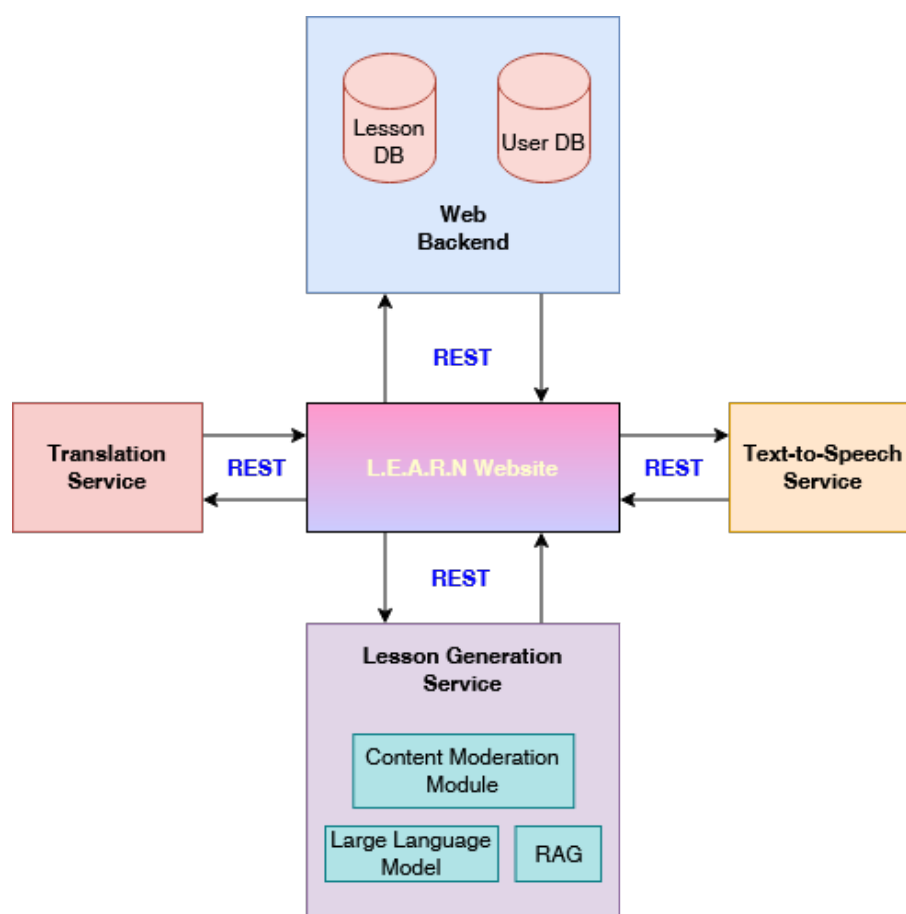


Figure 3.1: System architecture of the L.E.A.R.N platform.

At the center of the system is the **L.E.A.R.N Website**, which acts as the primary interface between users and the platform’s backend services. Designed for responsiveness and ease of use, the website allows users to enter topics, select preferred languages, navigate lessons, interact with quizzes, and engage with the integrated chatbot. It serves as a unified frontend that orchestrates communication with four major service layers via REST APIs:

- **Web Backend:** This service handles all authentication, session management, and persistent data storage. It interfaces with both the *Lesson DB*, which stores generated lesson content, and the *User DB*, which maintains user credentials, learning

history, and progress metrics. This ensures each user’s journey is personalized and consistently accessible across sessions.

- **Lesson Generation Service:** This is the intelligence hub of the system. Upon receiving a user-submitted topic, it first routes the input through a *Content Moderation Module* to validate relevance and safety. Valid topics are then processed using a *Large Language Model* (ChatGPT), guided by a *Retrieval-Augmented Generation (RAG)* pipeline that grounds the response in accurate and diverse external sources. The output is a well-structured, factually supported lesson tailored to the user’s input.
- **Text-to-Speech Service:** After the lesson is generated, this service converts the lesson content into high-quality audio using a neural voice synthesis model (e.g., Eleven Labs). The resulting audio is streamed back to the frontend, allowing users to listen to the material—an essential feature for users with visual impairments or auditory learning preferences.
- **Translation Service:** This service enables dynamic multilingual support across the platform. Whether translating lesson content, chatbot responses, or UI elements, it ensures users receive a consistent experience in their selected language. With support for over 30 languages, this service plays a key role in making L.E.A.R.N globally inclusive.

The modular nature of the architecture allows each service to scale independently, improving system efficiency and reliability. New functionalities—such as additional language packs, content formats, or accessibility tools—can be integrated without disrupting existing components. This architecture ensures that L.E.A.R.N remains adaptable, robust, and future-proof as a next-generation, AI-powered educational platform.

3.2 Core Functionality and Implementation

The core functionalities of our application include:

3.2.1 Dynamic Lesson Generation

At the heart of L.E.A.R.N lies its ability to create lessons dynamically, a feature that truly sets it apart from traditional learning platforms that rely on static, pre-written content. Rather than serving users a fixed set of modules, L.E.A.R.N generates personalized lessons on the fly, tailored specifically to the user’s selected topic, current understanding, and past interactions on the platform.

When a learner inputs a topic (e.g., “photosynthesis” or “ancient Greek philosophy”), the system doesn’t just fetch a pre-built lesson. Instead, it intelligently constructs a structured learning path using a large language model guided by **Retrieval-Augmented Generation (RAG)**. It starts by identifying the scope of the topic and breaking it down into a **Table of Contents** with subtopics that logically progress from foundational concepts to more advanced ideas.

Each section of the lesson is then generated in real time using **ChatGPT**, with factual context grounded in trusted sources like **Wikipedia**, **ArXiv**, and **Semantic Scholar** through our RAG pipeline. The system takes into account the user’s past quiz scores to determine the appropriate level of detail and complexity. For beginners, explanations are simplified. We used **reading level adaptation** via the **Dale-Chall readability formula** to achieve this functionality. For advanced learners, deeper insights and more technical definitions and vocabulary may be introduced.

Additionally, the lesson generation system incorporates formatting and design features to improve learning retention. Concepts are explained in bullet points, definitions are highlighted, and each lesson begins with a previous summary and an overview, and concludes with key takeaways and follow-up prompts or quiz hooks to keep the learner engaged.

This flexibility means no two users will ever experience the exact same lesson—even for the same topic—unless their learning profiles and preferences are identical. It also makes L.E.A.R.N ideal for both self-paced learners and structured academic supplementation, as it can regenerate or refine lessons as the learner grows.

In essence, **Dynamic Lesson Generation** transforms L.E.A.R.N from a content library into a living, adaptive tutor—one that understands what to teach, when to teach it, and how to teach it best for each individual learner.

3.2.2 RAG-Based Factual Checking

Retrieval-Augmented Generation (RAG) is a foundational component of our learning platform, designed to generate accurate, verifiable, and context-aware educational content. The RAG architecture addresses one of the key limitations of standalone language models—hallucination—by grounding all generated outputs in external, trusted sources of knowledge.

When a user inputs a topic, the system first encodes the query using Google’s Gemini Embeddings, producing a dense vector representation that captures its semantic meaning. This vector is then used to query a FAISS (Facebook AI Similarity Search) vector database, which contains pre-embedded text chunks from high-quality sources such as Wikipedia, ArXiv, Semantic Scholar, Project Gutenberg, and the Internet Archive. If

sufficiently relevant results are found within the FAISS index (based on semantic similarity), those chunks are passed as context to the language model.

However, if the FAISS search does not return relevant information—based on a similarity threshold—the system enters a fallback mode. In this case, it dynamically queries the external sources (e.g., Wikipedia, ArXiv, Semantic Scholar, etc.) to retrieve fresh content related to the user’s topic. The retrieved documents are then embedded on the fly using the same Gemini Embedding model. These new vectors are compared against the original query using cosine similarity, and only those that surpass a predefined relevance threshold are accepted. This two-tiered retrieval system—indexed vector search followed by dynamic external search—ensures both speed and depth of coverage, allowing the platform to handle both common and obscure topics with equal effectiveness.

Once the final set of relevant documents is assembled, they are concatenated with the user’s original query and sent to ChatGPT, the system’s generative model. ChatGPT uses this grounded context to produce content such as full lessons, outlines, clarifications, or trivia facts. To ensure transparency, the output includes clickable citations, allowing users to verify the source of each claim.

This architecture enables the platform to maintain high factual accuracy, minimize hallucination, and ensure that all generated educational material is grounded in authoritative sources. Embedding-based storage and retrieval play a critical role in this process by enabling semantic matching instead of relying on brittle keyword search. The integration of FAISS with real-time source querying and cosine similarity filtering gives the system both scalability and precision, making it robust enough to support a wide range of educational topics dynamically and reliably.

3.2.3 Multilingual Support

L.E.A.R.N was built to be inclusive across geographies and languages. Users can select their preferred language from a menu of 32 supported options. The platform translates not only the lesson content but also the interactive elements to maintain a consistent experience. Language preferences are stored in the user profile, so learners don’t need to reset their choice every time. This feature helps non-native English speakers engage deeply without the barrier of language complexity.

We take special care in ensuring the accuracy of technical and educational terminology. To provide high quality lesson content, we translate content using a two fold approach. First we translate the content using Google Translate API. Then to make sure some translations are not too literal and to keep in mind the intricacies of common lexicon, we do an additional pass through a large language model to make lessons seem more innate to users’ language preference. This double translation validation significantly reduces the likelihood of distortion in meaning.

Multilingual support isn’t limited to visual content. Our Text-to-Speech module, powered by Eleven Labs, also delivers high-quality spoken versions of lessons in the selected language, including regional accents and tone variations. Combined with multilingual chatbot capabilities which allow learners to interact in their native tongue, this functionality ensures that L.E.A.R.N not only speaks the language, but understands the learning needs in it.

3.2.4 Text-to-Speech (TTS)

To enhance accessibility and user engagement, our platform integrates a Text-to-Speech (TTS) module that converts dynamically generated lessons into high-quality audio. This feature is particularly valuable for users who prefer auditory learning, are visually impaired, or want to consume content hands-free. As the lessons are generated on the fly using ChatGPT, the TTS system seamlessly takes the finalized text output and synthesizes it into natural-sounding speech. To support our multilingual goals, the TTS system was designed to handle content in up to 32 different languages.

For the TTS functionality, we primarily relied on Eleven Labs, a leading voice synthesis API known for its high-quality, human-like audio output. Eleven Labs offers a robust set of neural voice models capable of producing expressive, natural speech across multiple languages. Its support for emotion, intonation, and accent variation made it a strong fit for our use case, where the tone and clarity of lesson delivery significantly impact user engagement.

Integration with Eleven Labs was handled via their cloud API. Once a lesson is generated by the system using ChatGPT, the finalized text is sent to the Eleven Labs TTS API, specifying the desired voice model and language. The API returns an audio stream, which is then embedded in the platform's frontend as a playable audio component. This process happens in real-time, ensuring that users can access the spoken version of the lesson with minimal delay.

By leveraging Eleven Labs, we were able to deliver consistent, high-quality speech across multiple languages without managing separate TTS models for each one. This integration not only supports learners with different preferences and needs but also aligns with our goal of building an inclusive, engaging, and universally accessible learning platform.

3.2.5 Interactive Learning Elements

The L.E.A.R.N platform emphasizes active engagement through a wide range of interactive learning elements, which are tightly integrated into each lesson. Rather than presenting static, text-heavy content, lessons are enriched with dynamically generated components such as quizzes, flashcards, graphs, timelines, map-based visualizations, drag-and-drop exercises, and definition-matching games. Each of these elements serves a distinct pedagogical purpose while enhancing the learner's ability to retain and apply new knowledge.

- **Quizzes** provide immediate assessment and feedback, allowing users to check their understanding and adjust their learning strategies accordingly.
- **Flashcards** help reinforce terminology and key facts, leveraging spaced repetition and active recall techniques to improve memory retention.
- **Graphs and Data Visualizations** make abstract or numerical information more concrete and interpretable, especially for subjects involving trends, comparisons, or statistics.
- **Timelines** support historical and sequential reasoning by visually mapping out chronological events or stages in a process.
- **Drag-and-Drop Activities** foster kinesthetic learning by allowing users to manipulate elements on the screen—useful for categorization, sequencing, or matching tasks.

- **Definition Matching** strengthens vocabulary and concept comprehension, especially when paired with visual or sentence-based clues.

These interactive modules are generated in real time, tailored to the user’s selected topic and language, and styled to be visually consistent with the rest of the platform. Their inclusion transforms passive content consumption into an active learning process, which is known to promote deeper understanding and long-term retention.

3.2.6 Content and Relevance Moderation

To preserve the educational integrity and safety of the L.E.A.R.N platform, we implemented a robust content and relevance moderation system that filters every user-submitted topic before it enters the lesson generation pipeline. Early in development, we noticed that vague, off-topic, or inappropriate user inputs—such as sensitive political issues, offensive terms, or open-ended prompts—often triggered poor generation results or misaligned lessons. This highlighted the need for a gatekeeping layer that could ensure input quality, preserve system resources, and maintain a focused learning environment.

Our moderation system performs two levels of filtering. First, we use the **Meta LLaMA Guard** model to detect unsafe, harmful, or sensitive content. LLaMA Guard is specifically optimized for safety-sensitive tasks and provides structured output for interpreting whether a prompt violates content policies. Second, we use **Google’s Gemini model** to assess topic relevance. Gemini was chosen for its strong language understanding and its ability to semantically classify whether a topic fits within supported academic or general knowledge domains. This dual-model setup allows the system to detect both safety violations and low-relevance or noisy input with high precision.

The moderation service is implemented as an independent, low-latency REST API that interfaces directly with the frontend. Upon detecting a flagged input, the system halts further processing and returns a friendly, multilingual prompt asking the user to revise their topic. This approach protects the backend generation pipeline, reduces hallucination risk, and improves the overall quality and trustworthiness of the generated lessons.

3.2.7 Trivia and Image Generation

To make the learning experience more engaging and interactive, our platform includes a feature that generates topic-relevant trivia facts and illustrative images while the lesson content is being processed. This feature was introduced to address a key user experience challenge: the short delay that occurs between topic selection and lesson delivery. Rather than showing a static loading screen, we use this time to keep the user intellectually engaged and entertained, thereby reducing perceived wait time and enhancing the overall usability of the platform.

When a user selects a topic, the system first sends a parallel request to our trivia generation module, which uses an LLM—such as GPT-4—to generate a short list of fun, factual, and relevant trivia based on the selected topic. The prompt is carefully structured to ensure that the trivia is concise, informative, and suitable for educational use. At the same time, an image generation request is triggered using a diffusion-based image model. For this, we employed APIs such as Replicate, specifically the FLUX Pro model, which generates visually relevant images that align with the lesson topic.

Both the trivia facts and the generated image are displayed to the user during the short processing window before the full lesson appears. This not only enriches the platform’s interactivity but also helps spark curiosity and provide contextual cues about the topic

ahead. Internally, these assets are not just visual or entertainment features—they act as learning warm-ups, orienting the learner toward the upcoming content in an informal, low-pressure manner.

By incorporating trivia and image generation, the platform maintains user attention and transforms passive wait time into an active, engaging preview experience. This small but impactful feature aligns with our broader goal of making learning enjoyable, immersive, and consistently stimulating.

3.2.8 Chatbot Integration

To support deeper understanding and provide on-demand assistance, our platform includes an integrated multilingual chatbot that functions as an intelligent learning assistant throughout the lesson experience. The chatbot is designed to respond contextually to the content and help users resolve confusion, clarify concepts, and ask follow-up questions in real time.

Users can trigger the chatbot by simply right-clicking on any word or sentence within the lesson they find difficult to understand. The chatbot operates in three distinct modes: **L.E.A.R.N**, which simplifies the lesson content into digestible explanations; **Clarify**, which defines complex terms and concepts; and **Questions**, where users can freely ask additional topic-related questions. This fine-grained control helps learners tailor the support they receive.

The chatbot is multilingual, supporting over 30 languages. It uses the same language model as our core lesson generation system (ChatGPT), enriched with guardrails to ensure responses remain factual, safe, and relevant. This turns the platform from a static content provider into a fully interactive and adaptive tutoring environment, empowering learners to explore material confidently and independently.

3.3 System Design and Implementation

3.3.1 System Design Overview

The backend is built using **Spring Boot**, where RESTful APIs expose endpoints to manage user authentication, lesson content generation, quiz tracking, and progress storage. The frontend, built in **React**, uses a component-based architecture to dynamically render lessons, quizzes, graphs, flashcards, and interactive modules based on user activity and lesson type. This separation of concerns ensures maintainability and scalability.

We follow a microservice-friendly monolithic design, with clear separation of modules like:

- **UserService:** Handles authentication and user retrieval.
- **LessonService:** Manages lesson generation and saving.
- **ProgressService:** Stores per-lesson and per-topic progress.

3.3.2 Algorithms and AI Integration

We integrated a Large Language Model (LLM) via API calls in **FastAPI** to generate:

- **Table of Contents (ToC):** Dynamically structured per topic, with content moderation checks using RAG to filter flagged or vague topics.
- **Lesson Content:** Structured into textual explanations, interactive elements, and quizzes.
- **Adaptive Learning Model:** Assesses quiz performance based on accuracy and completion time, determining progression. Proficiency score guides the next lesson's difficulty level (Easy, Medium, Hard).
- **Translation Model:** Supports multilingual delivery via a translation service API, enabling consistent lesson presentation in over 30 languages.

To simulate a digital tutor experience, each generated lesson includes:

- A quiz system
- Flashcards
- Visual graphs
- Takeaways and summaries
- Interactive games (e.g., typing challenges, memory match)
- References for deeper learning

Adaptive Learning Model Implementation

The adaptive model dynamically adjusts lesson difficulty based on quiz performance:

Metrics Used:

- **Accuracy:** Percentage of correct answers.
- **Completion Time:** Optional, but factored in as a speed-efficiency indicator.

Difficulty Thresholds:

- **Easy:** 0–50% proficiency
- **Medium:** 50–80%
- **Hard:** 80–100%

Proficiency Score Calculation:

$$ProficiencyScore = (Accuracy \times Weight_{Accuracy}) + (CompletionTimeFactor \times Weight_{Time})$$

Where:

- $Weight_{Accuracy}$ = e.g., 0.8
- $Weight_{Time}$ = e.g., 0.2
- CompletionTimeFactor is a normalized value (0 to 1)

This enables the platform to route users to the appropriate next lesson, tailoring content difficulty dynamically.

3.3.3 Framework Implementation

Frontend (React): Built using React’s component-based architecture with:

- **Component Structure:** Includes Body, TableOfContentsPage, LessonPage, Navbar, and reusable UI elements (e.g., buttons, indicators, loading animations).
- **State Management:** Managed via `useState` and `useLocation` hooks.
- **Routing:** Implemented using `react-router-dom`.
- **Styling:** Custom CSS modules ensure visual consistency.
- **Graphing:** `Recharts` library is used for data visualization.

Divergence from Vanilla Templates:

- Custom component hierarchy tailored for a learning platform.
- Dynamic content fetching from backend APIs (Spring Boot).
- Embedded adaptive learning logic and quiz/lesson rendering logic.
- Responsive and unique UI focused on learner engagement.

3.3.4 Backend Design and Uniqueness

The backend leverages JPA-based entity-repository mappings for:

- **Users**
- **UserLessonProgress:** Tracks which lessons are completed.
- **UserTopicProgress:** Stores the ToC the user was shown.

This ensures content persistence and reproducibility for returning users. All APIs follow RESTful principles, and authentication is managed via user email and session tokens stored in the frontend.

3.3.5 Unique Features and Design Decisions

- **Intelligent Content Delivery:** Dynamic generation and sequencing of lessons tailored to learner proficiency.
- **Seamless Multilingual Experience:** Translations embedded across the entire learning flow.
- **Interactive Learning:** Lessons integrate embedded quizzes, flashcards, graphs, and educational games.
- **AI + Persistence Hybrid:** Combines LLM generation with structured data tracking for consistent learner experiences.
- **Text-to-Speech Accessibility:** Audio lessons enhance access for auditory learners and visually impaired users.
- **Reference Generation using RAG:** Improves the factual grounding of generated content.
- **TOC-Driven Flow:** Lesson progression follows a structured, pre-generated table of contents.
- **Gamification:** Typing challenges, memory match games, and visual engagement features enhance retention.
- **In-Lesson Chatbot Support:** Context-aware chatbot accessible via right-click enables real-time help and follow-up queries.

3.3.6 Design Decisions Leading to Uniqueness

- **Backend-Driven Content Generation:** Centralizes lesson logic for flexibility and reuse.
- **API-Centric Frontend:** Clean separation of data handling (backend) and rendering logic (frontend).
- **Client-Side Routing:** Enables smooth, reload-free navigation between components.
- **Component-Based UI:** Modular architecture facilitates reuse and maintainability.

- **User-Centered UI Design:** Every design decision—from layout to animations—was made to support engagement and clarity.
- **Commitment to Accessibility:** Text-to-speech, visible navigation cues, and loading indicators make the platform inclusive.
- **In-Lesson Chatbot Integration:** Supports real-time learning reinforcement through contextual feedback and questioning.
- **Iterative Refinement:** UI logic and behavior (e.g., loading overlays, component props) were continuously debugged and refined to optimize usability and responsiveness.

Chapter 4

Retrospection

One of the most satisfying aspects of the project was seeing our dynamic lesson generation come to life. Initially, it was just an idea whose feasibility we were not sure of. From there, it evolved into a robust engine that not only tailors difficulty but does so in real time based on user performance and includes first-class support for features like text-to-speech, translation, and chatbot-based question answering. This required a deep integration of logic, backend support, and frontend interactivity—a task our team executed well.

The collaborative spirit of the team played a pivotal role in our success. During blockers and unexpected hurdles, such as redesigning UI components or debugging complex API integrations, each member stepped up beyond their assigned roles. This flexibility and sense of ownership helped us maintain momentum and morale.

In hindsight, more realistic sprint planning from the outset, along with earlier testing of external dependencies like TTS and translation APIs, could have saved us last-minute refactoring and design overhead. Nonetheless, we turned these challenges into valuable learning opportunities that will guide how we scope and stagger future projects.

Overall, our iterative approach helped us learn quickly, reprioritize smartly, and deliver a feature-rich educational tool that we are proud of. The process not only sharpened our technical skills but also taught us how to navigate ambiguity, collaborate effectively, and build by putting ourselves in the shoes of our users.

What Went Well

- Our team had strong collaboration throughout the development cycle, using tools like JIRA and Git effectively to track tasks and address blockers.
- The core functionalities like dynamic lesson generation, adaptive quizzes, and multilingual support were completed on time and exceeded expectations.
- The platform proved technically stable, and the addition of fun features like trivia, flashcards, and chatbot modes added polish and engagement.
- Despite lacking a dedicated UI/UX expert, our team pulled together to create a clean and accessible interface that users responded to positively.
- The use of Retrieval-Augmented Generation (RAG) with credible sources greatly reduced hallucinations and improved trust in the content.

What Could Have Gone Better

- The Talking Head avatar feature had to be dropped due to time and resource constraints, even though it was part of our initial non-priority roadmap.
- Translation accuracy across 32 languages was a challenge due to limited access to native speakers for testing, which forced us to rely on imperfect APIs.
- Early in Sprint 1, we underestimated the interdependency between lesson generation and adaptive quizzes, causing some scheduling reshuffles.
- The platform's initial visual design was subpar, and it took considerable effort and iteration to reach an acceptable aesthetic.
- Sprint planning could have been more accurate from the start. While we had strong collaboration tools, being a 4-member team meant facing tight constraints.

Chapter 5

Recommendations for future

Throughout the development of the L.E.A.R.N platform, each team member maintained personal notes on what worked well and what could be improved. These reflections were later distilled into shared team insights. Below, we summarize the most important process learnings and product directions we identified.

5.0.1 Team Process Learnings

- **Start Testing Dependencies Early:** External APIs like text-to-speech and translation models had hidden challenges and rate limits. Allocating time in early sprints specifically for testing critical third-party tools would improve reliability and reduce integration risk.
- **Overestimate Time for Frontend Work:** UI/UX tasks—especially those involving aesthetic polish—consistently took longer than expected. Future sprints should include buffer time for iterative frontend work to reduce pressure near deadlines.
- **Break Features into Smaller Chunks:** Complex stories, such as adaptive quizzes, often had multiple technical layers. Breaking them down into finer-grained subtasks in JIRA would improve visibility, parallelization, and task distribution.
- **Log More Dev Notes Along the Way:** Quick design decisions or debugging insights were sometimes lost. Maintaining informal developer journals in a shared workspace (e.g., Notion, Confluence) would enhance team alignment during rapid iteration phases.
- **Celebrate Milestones, Not Just Deadlines:** Recognizing small wins during weekly standups proved motivating. Institutionalizing this practice going forward can help maintain positive momentum and team morale.

5.0.2 Product Expansion Goals

In reflecting on the platform’s current state, we also identified several areas that we would expand upon given more time and resources:

- **Bring Back the Talking Head Avatar:** Initially envisioned to increase learner engagement—especially for younger users—this feature was deprioritized due to technical scope. In future versions, lightweight implementations using pre-rendered video or real-time animation libraries could revive this idea effectively.
- **Stronger Multilingual Validation:** Currently, the system relies heavily on automated translations. Partnering with native speakers, educators, or crowdsourced reviewers would significantly improve the trustworthiness and accuracy of lessons in key languages.
- **Mobile Optimization:** While the desktop experience is robust, mobile compatibility can be enhanced. Investing in responsive design improvements and potentially developing a mobile app would better serve users on mobile-first or low-cost devices.
- **Lesson Analytics and Insights:** Implementing a learner dashboard with personalized analytics, review suggestions, and knowledge gap indicators would take the adaptive experience to the next level and provide more autonomy to learners.

- **Plug-in for Classrooms and LMS Integration:** To expand into educational institutions, a version of L.E.A.R.N that integrates with classroom tools (e.g., Google Classroom, Moodle) and provides analytics for teachers could be highly impactful.
- **User Feedback Loops:** Simple feedback buttons or forms at the end of lessons and quizzes could help gather user sentiment, identify recurring issues, and guide future content or UX improvements.
- **Accessibility Beyond TTS:** Expanding accessibility options—such as high-contrast visual modes, dyslexia-friendly fonts, and keyboard-only navigation—would make the platform more inclusive for users with diverse needs.